# Informe sobre el proceso de solución — Monte Carlo con OpenMP

**Estudiante:** Andres Felipe Martinez Ortiz · **Código:** 2225112 · **Equipo:** Intel Core i5-12450HX (12 hilos lógicos), Windows con MSYS2

---

## 1. Decisiones de diseño

**Estructura del programa:** se conservo la estructura del programa secuencial (un ciclo `for` que cuenta aciertos del cuarto de círculo) y se le agregó `#pragma omp parallel for reduction(+:dentro) schedule(static)`. El número de hilos se elige con un segundo argumento de línea de comandos (`omp_set_num_threads`). El mismo archivo compila también sin `-fopenmp` y sirve de versión secuencial para medir $T(1)$.

**Generador de números aleatorios:** El enunciado pedía flujos independientes, sin solapamiento (semillas espaciadas al menos $N/p$ valores) y un resultado idéntico para cualquier $p$. `rand_r` no sirve: solo guarda 32 bits de estado y no permite saltar posiciones. Se uso SplitMix64 con salto directo, el estado $k$ es `SEMILLA + k·GAMMA` y el número $k$-ésimo es una mezcla de ese estado. La muestra $i$ toma las posiciones $2i$ y $2i+1$ de una única secuencia de periodo $2^{64}$, de modo que cada hilo recorre un tramo disjunto y el resultado no depende de cuál hilo ejecute cada muestra. Se eligió este generador, y no uno más sofisticado, porque es corto (una función de 5 líneas), lo puedo explicar completo y resuelve el problema del salto sin bibliotecas externas.

**Planificación.** Se fijo `schedule(static)`: todas las iteraciones cuestan lo mismo, así que un reparto en bloques iguales ya está balanceado.

## 2. Problemas que aparecieron y cómo los diagnostiqué

1. `schedule(runtime)` **seis veces más lento.** Al probar una versión parametrizable con `schedule(runtime)` y sin definir `OMP_SCHEDULE`, el programa con un hilo tardó 2,06 s frente a 0,34 s de la versión secuencial. Al fijar `OMP_SCHEDULE=static` volvió a 0,34 s; la causa es que libgomp usa por defecto `dynamic,1`. La evidencia fue comparar los tiempos con distintas planificaciones.
2. **Condición de carrera al quitar** `reduction`**.** Para entender II.4 se compilo una variante sin la cláusula. Con $p = 4$ los aciertos dieron 19 631 692 y 19 637 011

(frente a 78 538 546), es decir, $\hat{\pi} \approx 0{,}785$ en vez de 3,14, y distintos entre corridas. La evidencia que identifica una carrera es justamente esa, resultado erróneo y no reproducible.

3. **Resultado que dependiera de $p$.** El diseño por bloques o por hilo habría dado una estimación distinta para cada $p$ con lo cual se evito haciendo que cada muestra dependa solo de su índice.

## 3. Validación de la correctitud

- **Estimación de $\pi$:** con $N = 10^8$ se obtuvo $\hat{\pi} = 3{,}141542$, error absoluto $5{,}1 \times 10^{-5}$. La desviación típica teórica es $4\sqrt{q(1-q)/N} \approx 1{,}6 \times 10^{-4}$ con $q = \pi/4$, de modo que el error observado es de 0,3 desviaciones.
- **Reproducibilidad:** se ejecuto $p = 1, 2, 4, 8$ con la misma semilla y obtuve exactamente 78 538 546 aciertos en los cuatro casos. También lo verifiqué con $N$ impar y $p = 3$ (por ejemplo $N = 1001$: 797 aciertos con $p = 1$ y $p = 3$) .
- **Compilación:** sin advertencias con `gcc -O2 -Wall -Wextra -fopenmp montecarlo.c -o montecarlo -lm`.

## 4. Conexión con mis datos

La mediana de $T(1)$ fue 0,212 s y la de $T(4)$ 0,069 s, un speedup de 3,07 (eficiencia del 77 %). Con $p = 8$ la mediana fue 0,057 s. Pasar de 4 a 8 hilos solo ahorró 12 ms y el speedup subió a 3,72 (eficiencia del 46,5 %). El programa casi no usa memoria, así que el límite no es el ancho de banda; mis hipótesis son (i) la frecuencia por núcleo baja con más núcleos activos y (ii) ya que mi procesador es híbrido el `core id` muestra 4 núcleos con hiperprocesamiento y 4 sin él (núcleos P y E), y con `static` cada hilo recibe igual trabajo, por lo que el hilo que cae en un núcleo más lento fija el tiempo total. Con $p = 4$ se obtuvo $f \approx 0{,}899$ y Karp–Flatt $e = 0{,}101$, que sube a 0,164 con $p = 8$, parte de la pérdida son sobrecostos que crecen con $p$ y no una fracción secuencial fija. Con $p = 2$ los tiempos variaron entre 0,099 y 0,142 s, y por eso se uso la mediana (0,120 s) y no el promedio (0,123 s). `static` y `dynamic` dieron 0,053 y 0,051 s con $p = 8$, la diferencia no es significativa frente a la resolución de 1 ms.

## 5. Reflexión final

Aprendí que (a) en un problema "embarazosamente paralelo" lo difícil no es el ciclo sino los números aleatorios, hay que pensar el generador antes de paralelizar (b) una reproducibilidad independiente de $p$ se consigue haciendo que el resultado dependa del índice de la muestra y no del hilo y (c) un speedup bajo no siempre indica una parte secuencial, aquí casi no la hay, y el $f \approx 0{,}90$ es una fracción aparente que absorbe los sobrecostos y el hardware.

**Qué podría hacer diferente:** posiblemente aumentaría $N$ (por ejemplo a $10^9$) para que la ejecución dure segundos y los costos fijos y la resolución de 1 ms pesen menos; repetiría con más corridas; probaría $p = 6$ y $p = 12$ y fijaría la afinidad de hilos (`OMP_PROC_BIND`) para comprobar la hipótesis de núcleos P/E.

---

# Anexo: Bitácora de uso de IA generativa

**Herramienta usada en todas las consultas:** Claude (Anthropic), en la interfaz de chat.

| # | Propósito | Qué aportó | Qué acepté | Qué descarté / cambié | Razón |
|---|---|---|---|---|---|
| 1 | Entender el enunciado y planear el trabajo | Dividió el trabajo entre lo que corre en mi PC (mediciones) y lo que se redacta; hizo preguntas sobre semilla, generador y entorno | El plan y la idea de pedir mi semilla y sistema operativo | Nada | Entender el trabajo y todo lo que había en el antes de comenzar a trabajar |
| 2 | Diseño del generador | Explicó por qué `rand_r` no cumple las condiciones del enunciado y propuso bloques con SplitMix64 + xorshift | La idea de un generador propio | El esquema de bloques y xorshift: lo simplificó a SplitMix64 con salto directo por índice | Hacía falta algo que yo pudiera explicar y que diera el mismo resultado para todo $p$ |

| # | Propósito | Qué aportó | Qué acepté | Qué descarté / cambié | Razón |
|---|---|---|---|---|---|
| 3 | Escribir `montecarlo.c` | Código con `reduction` y `schedule(static)`; probó compilación con `-Wall -Wextra` y reproducibilidad para $p = 1, 2, 3, 4, 8$ | El código final | `schedule(runtime)`: por la lentitud de `dynamic,1` por defecto | Lo detecte midiendo |
| 4 | Scripts `bench.sh` y `grafica.py` | Medición automática (5 corridas, mediana) y gráfica de $S(p)$ | Ambos | Nada | Los ejecuté en mi PC y funcionaron |
| 5 | Borrador de respuestas (Partes I, II, III) | Redacción de I.1–I.3, II.3, II.4, III.1, III.5; cálculos de Amdahl y Karp–Flatt con mis datos | El borrador, que luego modifiqué con datos y resultados correspondientes a mi equipo. | modifique varias ideas de las respuestas para simplificarlas o explicarlas de mejor manera a mi punto de vista | Simplificar el documento, leerlo, entenderlo y no dejárselo todo a la IA |
| 6 | Interpretación de `cpuinfo.txt` | Identificó el patrón de procesador híbrido (núcleos P y E) a partir de los `core id` | Como hipótesis para explicar el speedup con $p = 8$ | Nada | Verificar la estructura de los `core id` en mi equipo personal |
| 7 | Borrador de este informe | Estructura y redacción | La estructura general del documento y las ideas principales | Solo cambios menores, el documento esta correcto | Optimizar el tiempo de trabajo |